

Architecture Design Document

Cloud Engineer Technical Assessment, Accor

Prepared by: Anurag Shrivastava

1. Summary

The Redemption is a business-critical microservice on AWS EKS that processes hotel point deductions for guests worldwide. Traffic is steady under normal conditions but can spike to ten times baseline during Flash Sales with no advance warning. Because the service affects customer loyalty balances directly, any downtime translates directly into lost revenue and lost customer trust.

This document sets out the architecture which will be used to keep the service available and responsive through sudden traffic spikes and infrastructure failures, the controls used to protect customer data and infrastructure integrity, and the operating model proposed for a three-person engineering team.

At a summary level, the platform scales itself automatically at both the application and infrastructure layers, is distributed across three Availability Zones so that no single failure removes capacity, applies defense in depth across the network and the cluster, and recovers from common failure modes (node loss, Availability Zone impairment, a bad deployment) without requiring a person to intervene first.

This document describes the complete target architecture, matching the accompanying diagrams.

2. Index

Sections 3 through 7 answer the requirements from the assessment brief directly, in the same order and using the same labels. Section 8 summarizes the trade-offs behind the major decisions. Section 9 lists assumptions made about items outside this repository's scope.

Requirement	Where it is answered
A. Compute & Architecture	Section 3
B. Scalability Strategy	Section 4
C. Security & Networking	Section 5
D. Reliability & Observability	Section 6
E. Operations (Day 2 Operations and Task Assignment)	Section 7

This document is one of three assessment deliverables and is meant to be read alongside the other two:

- Infrastructure as Code and Kubernetes manifests: Code/terraform and Code/kubernetes.
- Architecture diagrams: Diagram/Architecture-HA (single-region, high availability), Diagram/Architecture-HA-DR (multi-region, high availability and disaster recovery), and Diagram/Network Diagram (Availability Zone and subnet-level network layout).

3. Compute & Architecture

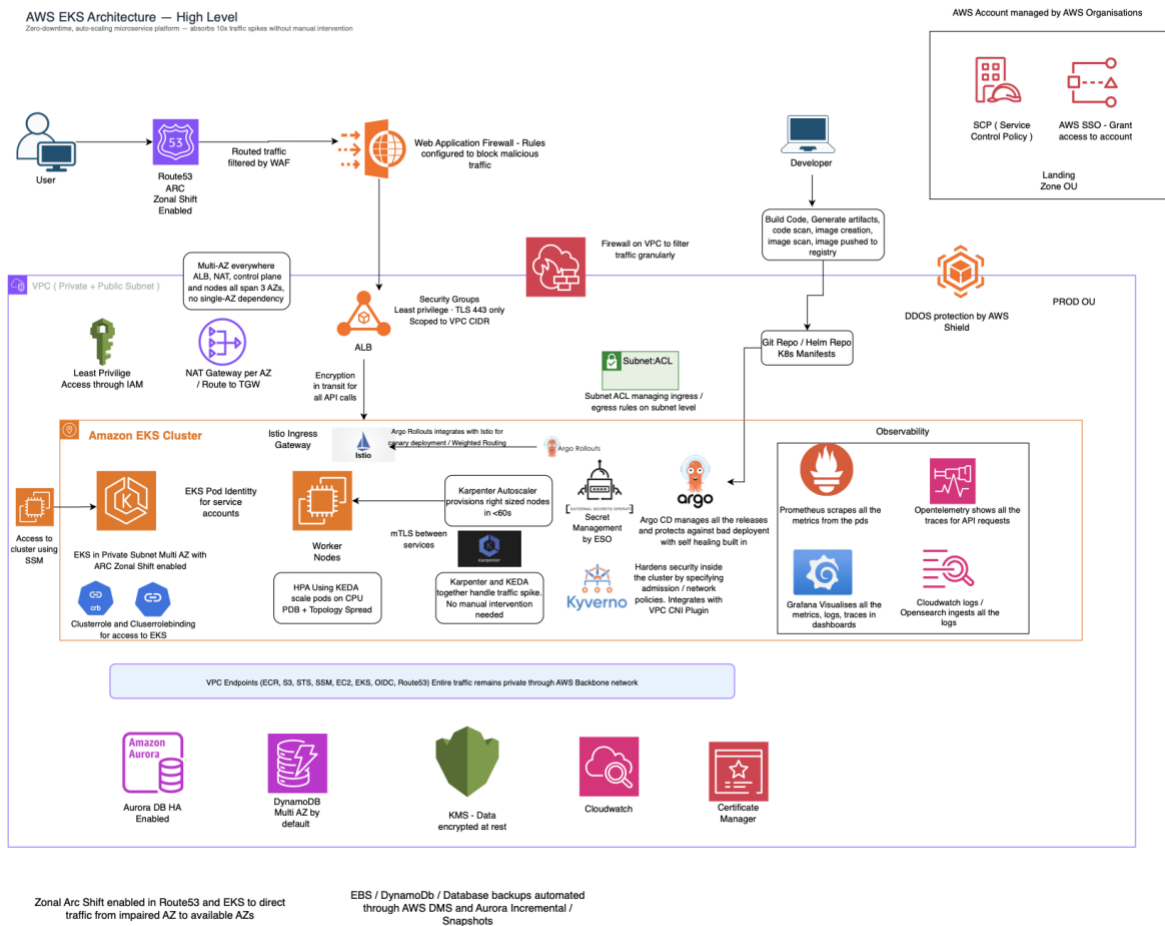


Figure 1. AWS EKS architecture, single-region high-availability view.

Assumption

It is assumed that the application is currently using a database with services running inside an EKS Cluster with KMS being used to encrypt data at rest.

Certificate manager being used to manage certificates and CloudWatch container insights being used for logging along with Prometheus & Grafana.

Network foundation

The cluster and its supporting infrastructure sit inside a dedicated VPC split into private and public subnets across three Availability Zones. Application workloads and platform services run exclusively in private subnets; nothing in the data path is directly reachable from the internet. Public subnets host only the internet-facing load balancer path and one NAT Gateway per Availability Zone, so a single NAT Gateway failure cannot remove egress capacity for the whole cluster. Eleven VPC interface and gateway endpoints, covering ECR, S3, STS, SSM, EC2, EKS, EKS Auth, the EKS OIDC issuer, Elastic Load Balancing, Auto Scaling, and Route 53, keep routine AWS API traffic on the AWS network backbone rather than the public internet.

Cluster and compute layout

The EKS control plane runs across all three Availability Zones with zonal shift enabled, so traffic from a single impaired zone can be moved to a healthy availability zone without manual action. Kubernetes Secrets are encrypted at rest using a dedicated customer-managed KMS key. The control plane also exposes a configurable scaling tier, so API server capacity can be increased ahead of a known event rather than only scaling the data plane.

Compute is deliberately split into two pools with different responsibilities:

- A static, fixed-size managed node group hosts platform components (Karpenter, Kyverno, Istio, KEDA, ArgoCD, Argo Rollouts, Vault) that need to be running before application capacity exists, and that should not compete with application pods for scheduling priority.
- Karpenter provisions and removes EC2 capacity on demand for the application workload itself, sized to whatever is actually pending rather than a pre-guessed instance count. It can handle both on-demand as well as spot instances.
- Nodes can be placed on warm pool so that they are available to accept traffic faster as compared to the time taken when the instance is booting up from scratch.

This split means the platform that manages scaling is not itself dependent on the thing being scaled, which matters most exactly when a Flash Sale starts and application demand climbs fastest.

Containerized application

The application is packaged as a Helm chart with a standard Kubernetes Deployment, Service, and ServiceAccount, resource requests and limits, liveness and readiness probes, a PodDisruptionBudget, and topology spread constraints that keep replicas balanced across all three Availability Zones. Availability during extreme load and infrastructure failure is addressed at every layer at once: enough replicas are always requested (Section 4), replicas are never concentrated in one zone or one node (this section and Section 6), and the load balancing and traffic management layer (Section 5) routes traffic only to healthy pods.

4. Scalability Strategy

Two independent control loops handle the Flash Sale scenario, one for application capacity and one for infrastructure capacity, and neither requires manual intervention.

Application-level scaling

The application chart supports two interchangeable scaling mechanisms, selected with a single value:

- A KEDA ScaledObject supports many different triggers eg. Kafka consumer lag, cron, storage queues etc. It has large support for scalers and is very extensible.
- A standard HorizontalPodAutoscaler, configured for 3 to 30 replicas targeting 60 percent average CPU utilization.
- A KEDA ScaledObject with the same 3 to 30 replica range, which can scale on CPU as configured today or be pointed at other signals, such as request rate or queue depth, without changing the underlying Deployment.

Both paths scale the same Deployment, so the scaling trigger can change without changing how the application is packaged or deployed.

Infrastructure-level scaling

Karpenter watches for pods that cannot be scheduled and provisions right-sized EC2 capacity for them directly, typically in under a minute, instead of waiting for a pre-configured Auto Scaling Group to catch up. The node pool is restricted to on-demand capacity in current-generation compute, memory, and general-purpose instance families, with a cluster-wide limit of 1000 vCPU as a cost and blast-radius guardrail. When demand drops, Karpenter consolidates and removes idle nodes after one minute of low utilization, so capacity and cost contract automatically once the Flash Sale ends. Nodes are also recycled every 30 days regardless of load, which keeps the underlying AMI and packages current without a separate patching process.

Together, these two loops mean the system's response to a Flash Sale is the same as its response to an ordinary Tuesday: watch demand, react automatically, and settle back down afterward.

5. Security & Networking

Edge

An internet-facing Application Load Balancer, fronted by AWS WAF and protected by AWS Shield, is the intended entry point for customer traffic. Route 53, together with Application Recovery Controller, handles DNS-level routing and zonal or regional failover. TLS termination is disabled so that traffic is encrypted end to end. Once inside the cluster the communication of services within the cluster is also encrypted using mTLS.

Network

The VPC separates public and private subnets across three Availability Zones. Security groups are scoped to the minimum required traffic; for example, the cluster security group allows inbound TLS on port 443 only, from the VPC CIDR / Subnet CIDR Ranges. Network ACLs at the subnet boundary are part of the intended design as a second, stateless layer behind security groups. VPC endpoints keep AWS API traffic off the public internet entirely.

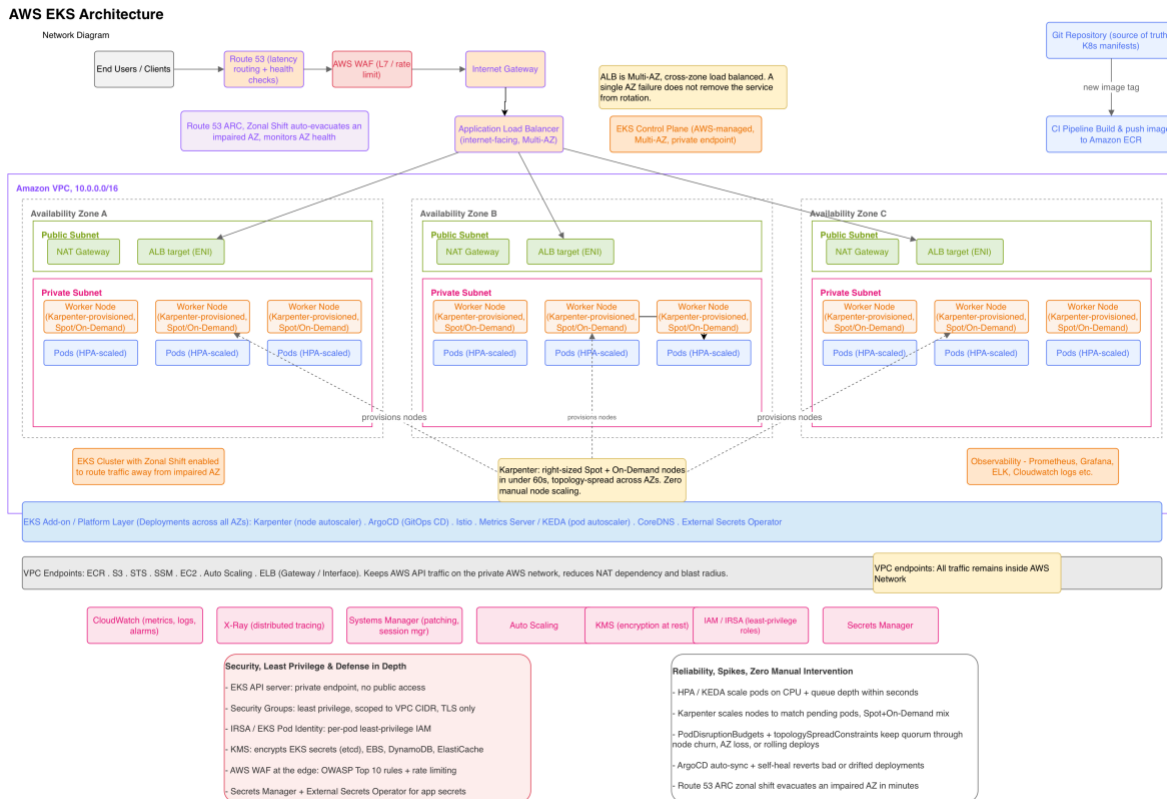


Figure 3. AWS EKS network diagram: Availability Zone, subnet, and traffic-flow view, referenced in Section 5.

Cluster and workload

Kyverno is deployed as a policy engine to enforce baseline pod security standards and other admission-time rules automatically, without a person reviewing every manifest by hand. Istio provides the data plane for mutual TLS between services and the traffic splitting that Argo Rollouts uses during canary releases.

Identity and access

IAM follows least privilege throughout. EKS Pod Identity is used instead of long-lived credentials, and controller permissions are scoped narrowly rather than granted broadly. As one concrete example, Karpenter's controller role can only create and tag the specific EC2 resource types it needs, can only terminate instances and delete launch templates that carry its own node pool's tag, and can only pass one specific node IAM role, to the EC2 service specifically. This is intentionally narrower than attaching a general EC2 administrator policy.

Secrets and encryption

HashiCorp Vault, deployed in high-availability Raft mode across three replicas, is the central store for application and platform secrets. Its Agent Injector delivers secrets to pods directly, without embedding them in manifests or images. A customer-managed KMS key encrypts Kubernetes Secrets at rest, alongside the data layer's own encryption at rest.

To extend the functionality External Secrets Operator can be used to integrate with different providers and manage secrets efficiently using a single operator.

6. Reliability & Observability

Health signals

Liveness and readiness probes on every application pod are the first health signal, feeding directly into Kubernetes' own self-healing: a failing pod is restarted, or removed from Service endpoints, automatically. The target platform wide signal set layers three kinds of telemetry on top of that: metrics (Prometheus, visualized in Grafana), traces (OpenTelemetry, to identify which specific service in a request path is slow), and logs (CloudWatch Logs and OpenSearch). Jaeger can be used to visualize the traces and they can also be exported to Tempo / Elasticsearch / OpenSearch.

Recovery by failure type

Node loss: Kubernetes reschedules affected pods onto healthy nodes automatically. The PodDisruptionBudget (a minimum of 2 of 3 replicas available) and the Availability Zone topology spread constraint ensure this does not concentrate the remaining replicas in a single zone or drop below a safe minimum while it happens.

Availability Zone impairment: EKS zonal shift removes an impaired zone from rotation without waiting for a person to notice and act. The design extends this to DNS-level routing through Route 53 Application Recovery Controller for a full regional failure, illustrated in the HA-DR variant of the architecture diagram.

Bad deployment. ArgoCD's continuous reconciliation and self-healing revert unauthorized or drifted changes automatically. The target path layers Argo Rollouts canary analysis on top of this, shifting a small percentage of traffic to a new version and rolling back automatically if it fails health checks, rather than exposing all users to a bad release at once.

In every case, the objective is the same: the system detects and reacts to the problem faster than an on-call engineer could be paged.

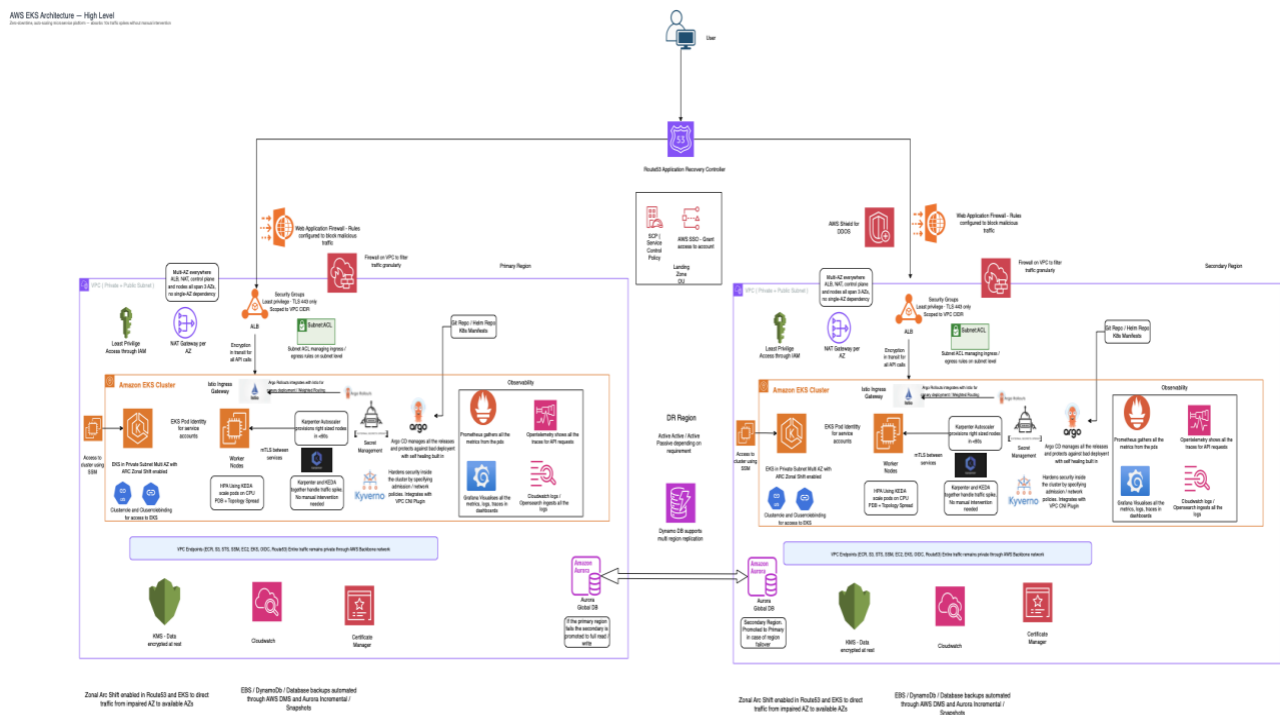


Figure 2. Multi-region high-availability and disaster-recovery variant, referenced in Sections 6 and 8.

7. Operations

7.1 Day 2 Operations

Reducing operational toil was a design input, not an afterthought. The following mechanisms specifically remove recurring manual work:

- GitOps through ArgoCD removes manual deployment steps; engineers change a file in Git, and the cluster converges to match it.
- Karpenter and KEDA remove manual capacity planning; nobody needs to forecast Flash Sale traffic and pre-provision servers for it.
- Kyverno removes manual security review for common misconfigurations by catching them automatically at admission time.
- Zonal shift removes the need for a manual, out-of-hours failover during an Availability Zone impairment.
- Centralizing metrics, logs, and traces in one observability stack is intended to replace hunting across several tools during an incident with a single starting point.

Further to initial automations to reduce toil, there are also BAU activities:

- Maintenance and upgrades to infrastructure such as EKS cluster upgrades, database upgrades, version updates for container images and patching for vulnerabilities.
- Scheduled backups and disaster recovery drills, periodic failover testing and chaos or fault injection testing.
- Secrets rotation and RBAC reviews.
- Cost optimization and review of the infrastructure including right sizing of instance families.
- On-call support as required, defining service level objectives (SLOs), error budgeting and conducting blame-less post-mortems.
- Runbook maintenance.

7.2 Team Delegation Plan

The team consists of one Senior Engineer and two Junior Engineers. Work is divided so the Senior Engineer owns the areas with the highest cost of getting wrong (networking, identity and access, and core cluster configuration), while the two Junior Engineers own well-scoped, independently reviewable tracks that do not block each other.

The Senior Engineer also mentors the Junior Engineers so that everyone in the team is capable of handling the environment.

Owner	Responsibilities
Senior Engineer	Overall architecture and diagram ownership; VPC and network design; EKS cluster and least privilege IAM design; Karpenter scaling strategy; security review of all Junior Engineer work; final sign-off before any change is considered complete.
Junior Engineer 1: Deployments Track	ArgoCD and Argo Rollouts configuration; the application Helm chart; KEDA scaling rules; Kyverno policies (reviewed by the Senior Engineer before merge).
Junior Engineer 2: Reliability & Observability	Prometheus, Grafana, and OpenTelemetry setup; CloudWatch and OpenSearch logging; PodDisruptionBudget tuning; validating zonal shift behavior; documenting Availability Zone impairment and bad-

Track	deployment recovery runbooks.
-------	-------------------------------

8. Key Architectural Decisions & Trade-offs

No architectural decision is free. The table below lists the major choices made for this system and what was given up in exchange.

Decision	Rationale	Trade-off
Private subnets with a NAT Gateway per Availability Zone, rather than a fully isolated or fully public design	Keeps every workload off the public internet while still allowing image pulls and calls to external services	NAT Gateways carry an hourly and per-gigabyte cost. A fully private design using only VPC endpoints would be cheaper but could not reach public container registries. Alternate would be to use TGWs. That would also depend on the network infrastructure of the organization if everything sits behind a corporate proxy.
Karpenter, rather than a fixed-size node group or Cluster Autoscaler, for application capacity	Provisions the right size and number of nodes within roughly a minute of demand appearing, instead of pre-guessing capacity	A newer and more complex tool than a fixed node group; the team needs to understand its instance-selection and consolidation behavior.
KEDA, offered alongside a standard HPA, rather than relying on CPU-based scaling alone	Can scale on real signals such as queue depth or request rate, which fits a spiky Flash Sale pattern better than CPU alone	One more moving part to operate and monitor, alongside the HPA path the chart also supports. It would also require performance testings to gather which metric to look for.
Istio service mesh	Provides service-to-service mTLS and fine-grained traffic control for canary releases without application code changes	Adds real operational complexity and a small amount of latency per request of about 2ms.
Kyverno for policy enforcement	Blocks insecure configurations automatically, before they can cause an incident	Policies must be authored and maintained, and an overly strict policy can block a legitimate deployment.
External Secrets Operator / HashiCorp Vault, rather than AWS Secrets Manager	A single place to manage secrets for both AWS and non-AWS systems, with fine-grained access policies	Vault is self-managed here, so the team owns its uptime, unsealing, and upgrades.
Zonal shift (EKS,	Redirects traffic away from an	Protects against a single

extending to Route 53 Application Recovery Controller) for Availability Zone failures	unhealthy Availability Zone automatically, within minutes and without manual failover	Availability Zone failing, not a full regional outage. A second AWS region, shown in the HA-DR variant of the architecture diagram, would be needed for that, at meaningfully higher cost and complexity.
---	---	---

9. Assumptions & Out-of-Scope Items

This repository addresses the workload-level architecture for The Redemption. The following are assumed to already exist as part of an AWS Landing Zone and are intentionally outside this repository's scope: the AWS Organizations structure, Service Control Policies, and IAM Identity Center (AWS SSO) account access shown in the architecture diagram.

10. Improvements

The security of this infrastructure can be further enhanced by implementing a two-account approach. The first account hosts all network components while the second hosts the entire backend. Traffic between the accounts can be managed using a private endpoint service or a vpc lattice service.

An API Gateway, a managed AWS service, can be used instead of an ALB. It provides features such as rate limiting, throttling and circuit breaking. However, it introduces additional considerations such as managing two accounts and challenges with disaster recovery and failovers which can be addressed with further analysis and solutions.